

Rachunek Macierzowy

Projekt 1

Natalia Bratek, Joanna Konieczny

1. Treść projektu

Dla macierzy o rozmiarze mniejszym lub równym $2^l \times 2^l$ algorytm tradycyjny. Dla macierzy o rozmiarze większym od $2^l \times 2^l$ algorytm rekurencyjny Bineta.

2. Opis

Chcemy wyznaczyć macierz C , będącą iloczynem macierzy A i B , czyli $C = A \cdot B$. Algorytm Binet'a polega na rozbiciu macierzy na mniejsze bloki, a następnie mnożeniu tych bloków i sumowaniu

ich zgodnie z zasadami mnożenia macierzy. Po wykonaniu obliczeń dla wszystkich bloków, wyniki są łączone w macierz C . Implementacja algorytmu Bineta została rozszerzona tak, aby działała również dla macierzy o dowolnych wymiarach, nie tylko będących potęgą liczby 2. W przypadku nieparzystych wymiarów stosowany jest mechanizm dynamic peeling, który polega na odseparowaniu dodatkowego wiersza lub kolumny i uwzględnieniu ich wkładu w końcowym wyniku. Dodatkowo dla macierzy prostokątnych stosowane jest rozszerzenie macierzy do rozmiaru kwadratowego poprzez dopełnienie zerami, a po obliczeniach wynik jest przycinany do oryginalnych wymiarów.

Another approach, called **dynamic peeling**, deals with odd dimensions by stripping off the extra row and/or column as needed, and adding their contributions to the final result in a later round of fixup work. More specifically, let A be an $m \times k$ matrix and B be a $k \times n$ matrix. Assuming that m , k , and n are all odd, A and B are partitioned into the block matrices

$$A = \left(\begin{array}{c|c} A_{11} & a_{12} \\ \hline a_{21} & a_{22} \end{array} \right) \quad \text{and} \quad B = \left(\begin{array}{c|c} B_{11} & b_{12} \\ \hline b_{21} & b_{22} \end{array} \right),$$

where A_{11} is an $(m-1) \times (k-1)$ matrix, a_{12} is a $(m-1) \times 1$ matrix, a_{21} is a $1 \times (k-1)$ matrix, a_{22} is a 1×1 matrix and B_{11} is an $(k-1) \times (n-1)$ matrix, b_{12} is a $(k-1) \times 1$ matrix, b_{21} is a $1 \times (n-1)$ matrix, b_{22} is a 1×1 matrix. The product $C = AB$ is computed as

$$\left(\begin{array}{c|c} C_{11} & c_{12} \\ \hline c_{21} & c_{22} \end{array} \right) = \left(\begin{array}{c|c} A_{11}B_{11} + a_{12}b_{21} & A_{11}b_{12} + a_{12}b_{22} \\ \hline a_{21}B_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{array} \right), \quad (9)$$

where $A_{11}B_{11}$ is computed using Strassen's algorithm, and the other computations constitute

3. Pseudokod

- tradycyjne mnożenie macierzy

n – liczba wierszy A
 k – liczba kolumn A

```

m – liczba kolumn B
Utwórz macierz C o wymiarach (n × m) wypełnioną zerami
dla i od 0 do n-1:
    dla j od 0 do m-1:
        dla t od 0 do k-1:
            C[i, j] = C[i, j] + A[i, t] * B[t, j]
zwróć C

```

- algorytm Bineta

```

binet(A, B, l)
    Jeśli rozmiar macierzy (n) ≤ 2^l
        wykonaj tradycyjne_mnożenie_macierzy(A, B)
    Jeśli rozmiar macierzy A jest nieparzysty
        wykonaj podział macierzy A i B na podmacierze dynamiczne
        A11, A12, A21, A22 oraz B11, B12, B21, B22
        o rozmiarach (n-1) × (n-1), (n-1) × 1, 1 × (n-1), 1 × 1
        odpowiednio, gdzie n to rozmiar macierzy A i B
        oblicz pomocnicze macierze:
            C11 = binet(A11, B11) + standardowe_mnożenie(A12, B21)
            C12 = standardowe_mnożenie(A11, B12) +
            standardowe_mnożenie(A12, B22)
            C21 = standardowe_mnożenie(A21, B11) +
            standardowe_mnożenie(A22, B21)
            C22 = standardowe_mnożenie(A21, B12) +
            standardowe_mnożenie(A22, B22)
        zwróć połączone macierze C11, C12, C21, C22

    W przeciwnym przypadku (n jest parzyste):
        wykonaj podział macierzy A i B na równe 4 bloki: A11, A12,
        A21, A22 oraz B11, B12, B21, B22

        oblicz pomocnicze macierze:
            C11 = binet(A11, B11) + binet(A12, B21)
            C12 = binet(A11, B12) + binet(A12, B22)
            C21 = binet(A21, B11) + binet(A22, B21)
            C22 = binet(A21, B12) + binet(A22, B22)

        zwróć połączone bloki (C11, C12, C21, C22)

```

4. Fragment kodu

- algorytm Bineta

```

def __binet(self, A, B):
    n = len(A)
    if n <= 2 ** self.l:

```

```

        return self.calc.traditional_matrix_multiplication(A, B)

    if n % 2 == 1:
        A11, A12, A21, A22 =
self.calc.split_into_block_matrices_dynamic_peeling(A)
        B11, B12, B21, B22 =
self.calc.split_into_block_matrices_dynamic_peeling(B)

        C11 = self.calc.add(self.__binet(A11, B11),
self.calc.standard_matrix_multiplication(A12, B21))
        C12 =
self.calc.add(self.calc.standard_matrix_multiplication(A11, B12),
self.calc.standard_matrix_multiplication(A12, B22))
        C21 =
self.calc.add(self.calc.standard_matrix_multiplication(A21, B11),
self.calc.standard_matrix_multiplication(A22, B21))
        C22 =
self.calc.add(self.calc.standard_matrix_multiplication(A21, B12),
self.calc.standard_matrix_multiplication(A22, B22))

    return
self.calc.connect_block_matrices_dynamic_peeling(C11, C12, C21, C22)
    else:
        A11, A12, A21, A22 =
self.calc.split_into_block_matrices(A)
        B11, B12, B21, B22 =
self.calc.split_into_block_matrices(B)

        C11 = self.calc.add(self.__binet(A11, B11),
self.__binet(A12, B21))
        C12 = self.calc.add(self.__binet(A11, B12),
self.__binet(A12, B22))
        C21 = self.calc.add(self.__binet(A21, B11),
self.__binet(A22, B21))
        C22 = self.calc.add(self.__binet(A21, B12),
self.__binet(A22, B22))

    return self.calc.connect_block_matrices(C11, C12, C21, C22)

```

- tradycyjne mnożenie macierzy

```

def traditional_matrix_multiplication(self, A, B):
    assert A.shape[1] == B.shape[0], "Wrong shapes"
    n = A.shape[0]
    k = A.shape[1]
    m = B.shape[1]
    C = np.zeros((n, m))

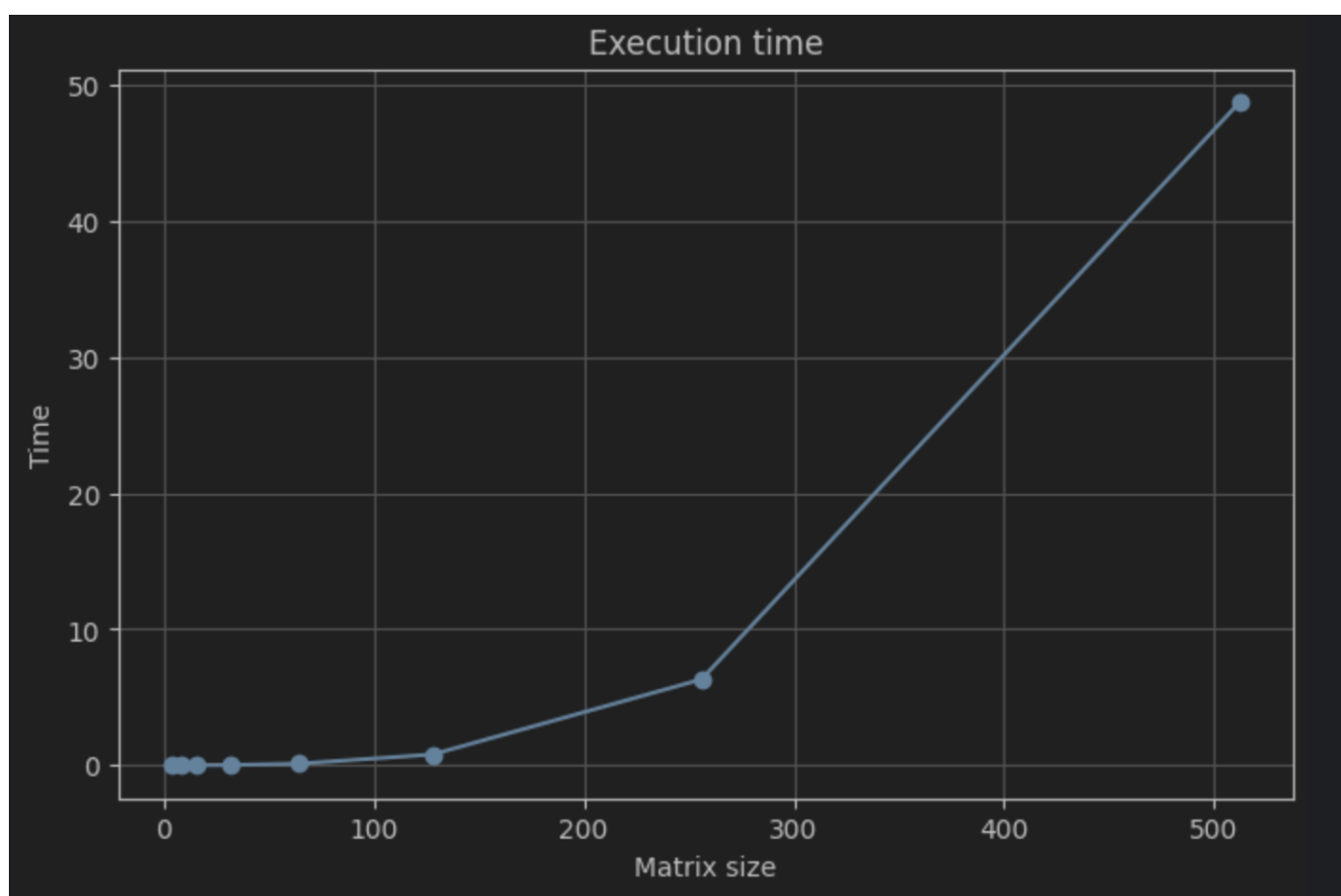
```

```
for i in range(n):
    for j in range(m):
        for t in range(k):
            C[i, j] += A[i, t] * B[t, j]
            self.multiply_count += 1
            self.total_count += 1
            if t > 0:
                self.add_count += 1
                self.total_count += 1

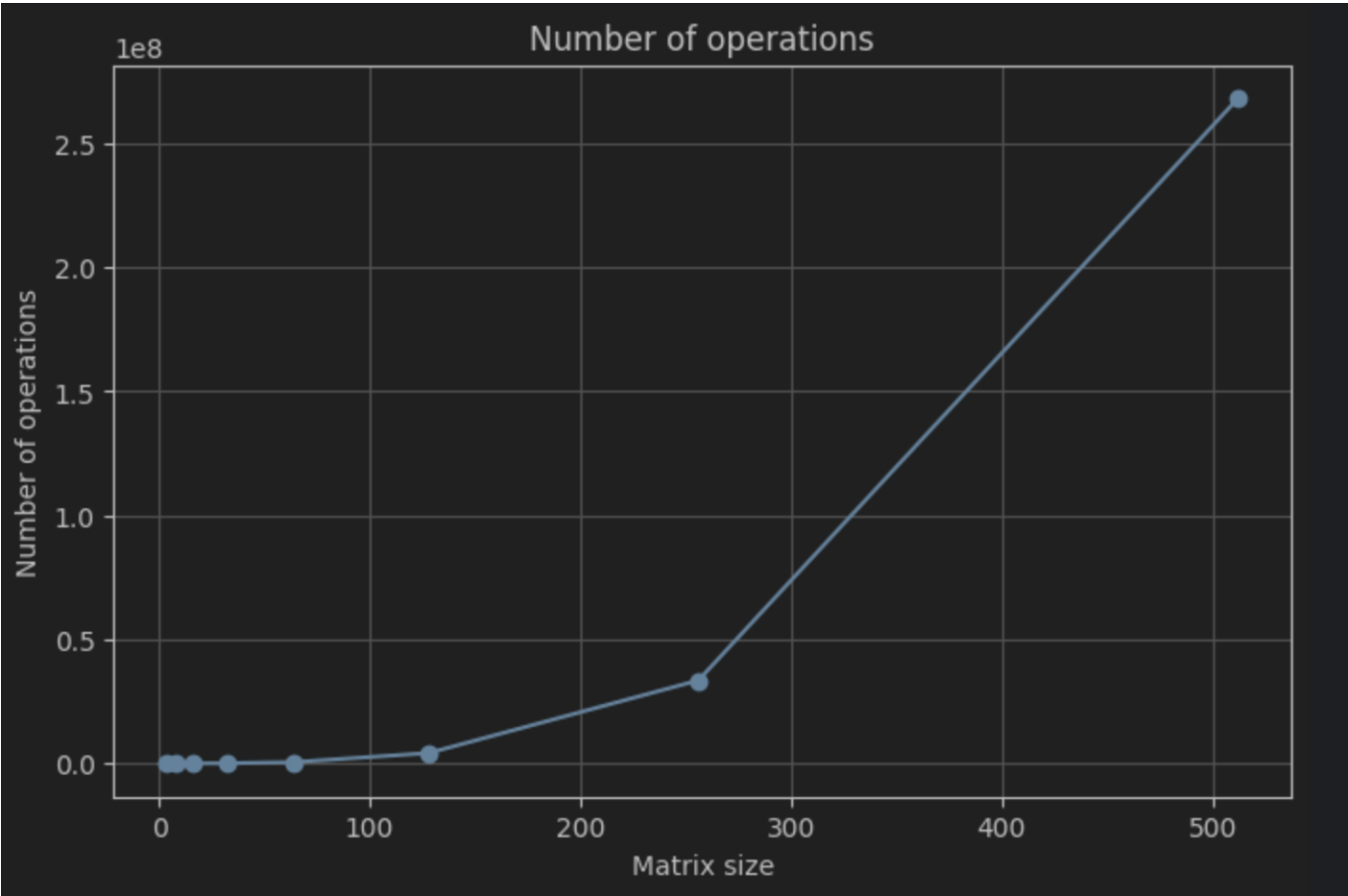
return C
```

4. Wyniki

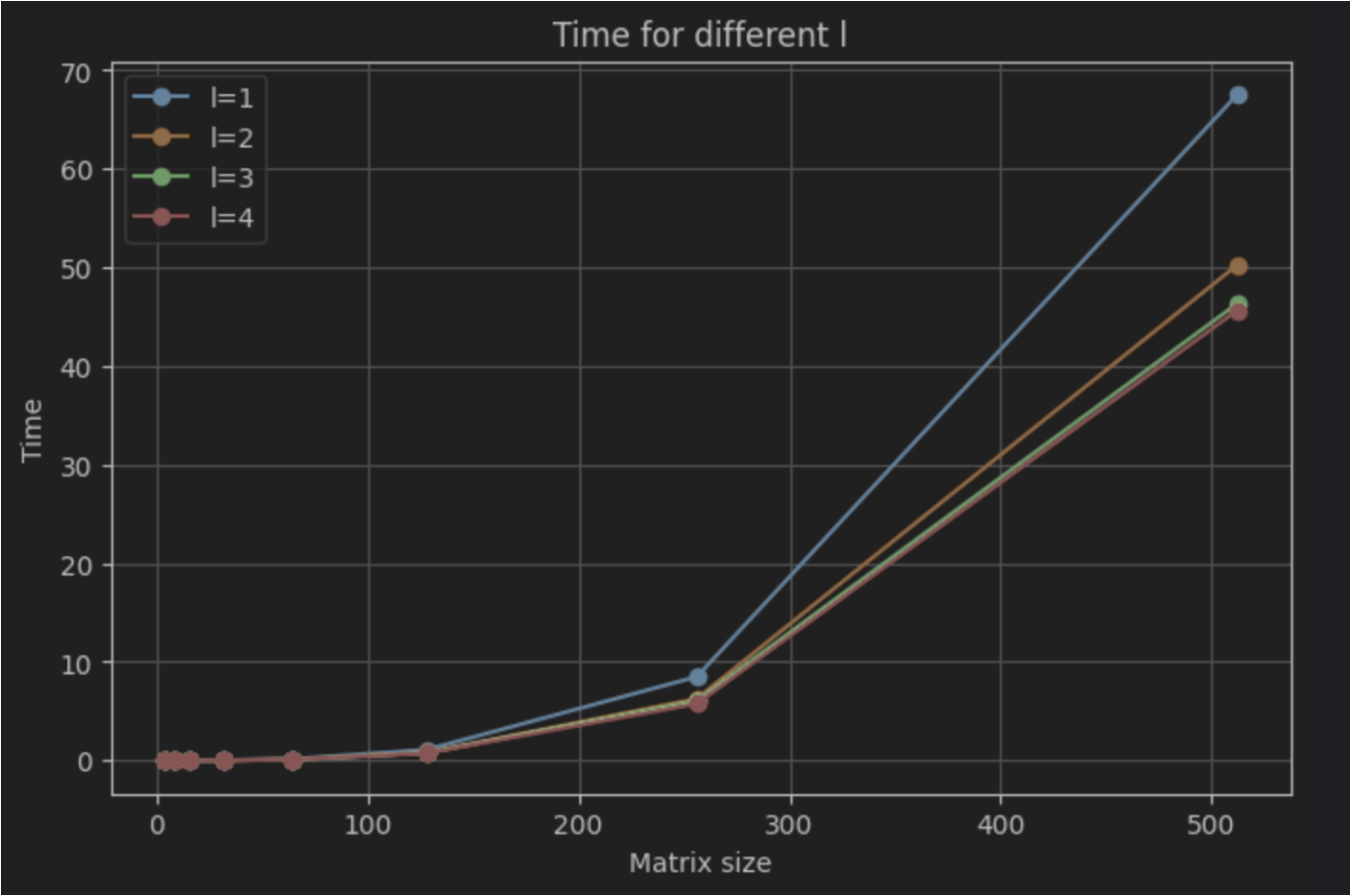
- wykres czasu wykonania od rozmiaru macierzy



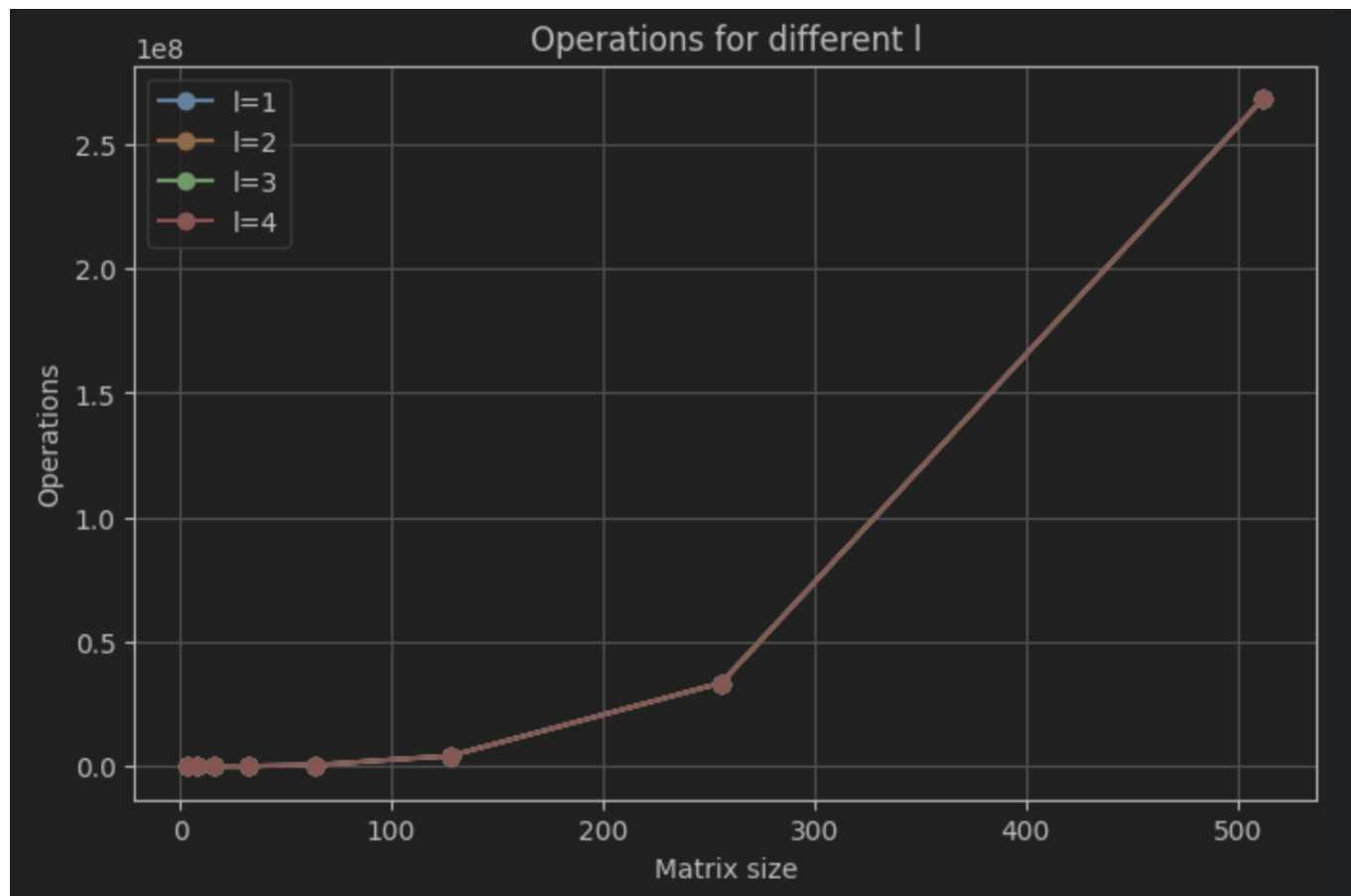
- wykres liczby operacji od rozmiaru macierzy



- wykres czasu wykonania od rozmiaru macierzy dla różnych wartości parametru l



- wykres liczby operacji od rozmiaru macierzy dla różnych wartości parametru l



5. Wnioski

- czas i liczba operacji rosną w przybliżeniu jak $O(n^3)$
- algorytm Bineta nie jest szybszy asymptotycznie od klasycznego mnożenia macierzy
- im większe l tym mniej poziomów rekurencji i krótszy czas wykonania
- dla małych macierzy różnice między wartościami l są pomijalne
- dla dużych macierzy wybór l ma zauważalny wpływ na wydajność